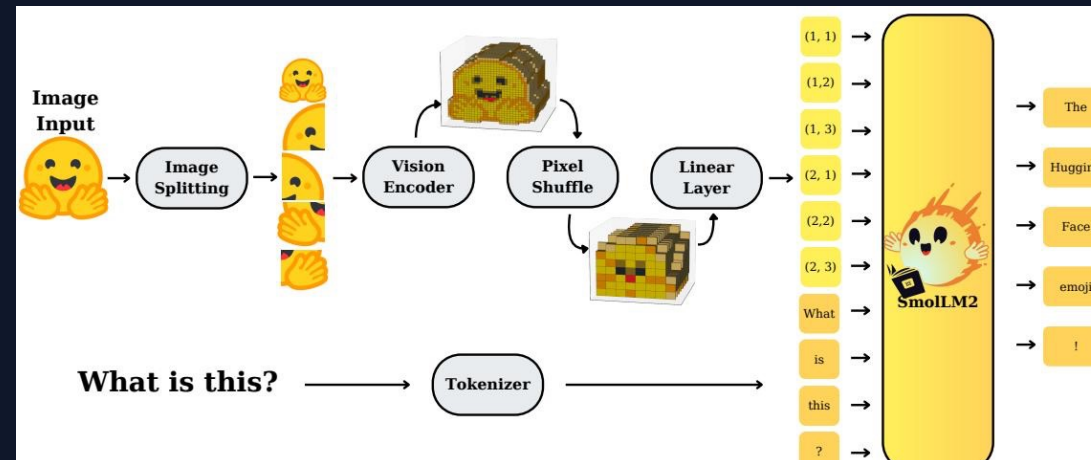


SmolVLM: Redefining Small and Efficient Multimodal Models

Compact VLMs for mobile and edge inference



Architecture evidence: image splitting → SigLIP encoder → pixel shuffle → projection → SmolLM2

256M 0.8GB RAM

Sub-1GB inference

500M 1.2GB RAM

Compact middle tier

2.2B 4.9GB RAM

16K context variant

Hugging Face + Stanford • fully open weights, datasets, code, and HuggingSnap mobile app

Large VLMs Are Powerful but Impractical for Edge Deployment

The deployment bottleneck is not capability alone—it is memory, latency, and local availability.



Edge target

phones • laptops • low-power devices

Best Efficient OS baseline

Molmo-A1B-7B 27.7GB RAM

too large

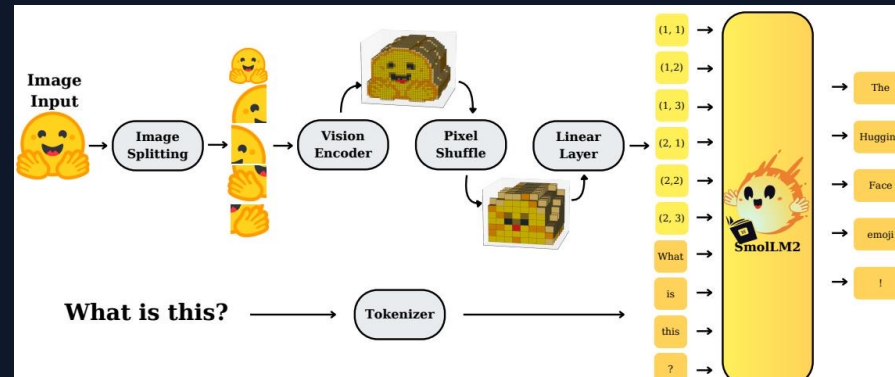
SmoIVLM-256M

0.8GB RAM batch=1

- Massive compute and memory limit mobile/edge VLM use.
- SmoIVLM-256M runs inference below 1GB GPU RAM.
- Smallest model outperforms 300× larger Llafacs-80B per paper key finding.
- All weights, datasets, code, and mobile app are open-sourced.

Design question: how much vision compute, context, and visual-token budget does a compact VLM actually need?

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone



1 Split image

Images are split into sub-images.

2 Encode

SigLIP vision encoder processes visual features.

3 Compress

Pixel shuffle reduces visual token count.

4 Project + fuse

Linear projection maps into SmolLM2 input space; text embeddings concatenate.

Video follows the same recipe: sample frames, process visual tokens, then concatenate with language tokens.

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

For small LMs, spending parameters on a large vision encoder is not automatically better.

Vision encoder parameter budget

SigLIP-B/16



93M

wins with 135M LM

SigLIP-SO400M



428M

larger, underperforms at smallest LM

428M

Experimental finding

93M encoder + 135M LM

outperforms

428M encoder + same LM

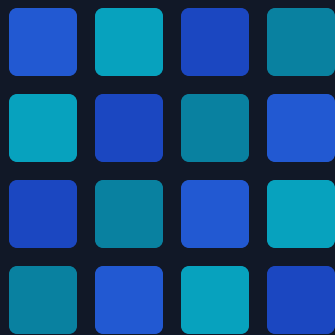
- Compact VLMs benefit from balanced encoder–LM compute.
- Only at 1.7B LM scale does the larger encoder justify its 10% parameter overhead.

**Design principle: allocate visual compute proportional to language model capacity—
not maximum encoder size.**

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

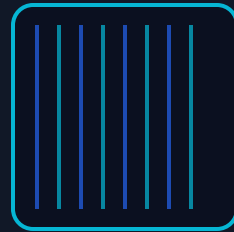
Pixel shuffle rearranges spatial features into channels; token count falls by r^2 .

Space-to-depth compression



16 spatial tokens

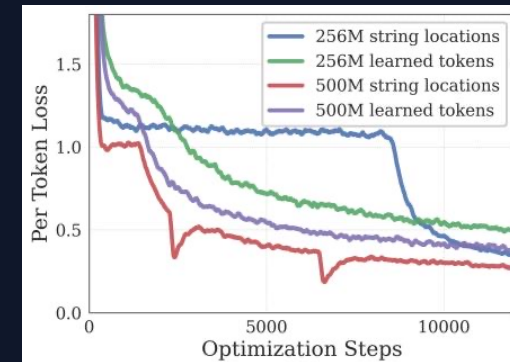
$r = 4$



1 token

with more channels

$$\text{Token reduction} = r^2 = 4^2 = 16\times$$



Source pixel shuffle diagram

- Small VLMs benefit from $r=4$ compression.
- Larger models use $r=2$ in the reported design finding.
- Lower attention overhead helps long-context modeling.

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Longer context lets the model preserve more visual evidence after image splitting and compression.



Recipe used in SmolVLM

RoPE base 10K → 273K

Fine-tuning data mixed long/short context

Model allocation 2.2B uses 16K; smaller variants use 8K

Finding: performance improves significantly when context extends from 2K to 16K tokens.

SmoIVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Capability	Benchmark	256M	500M	2.2B	Best Efficient OS baseline
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7 Molmo-A1B-7B
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0 Molmo-A1B-7B
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0 Molmo-A1B-7B
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5 Molmo-A1B-7B
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7 Molmo-A1B-7B
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5 Molmo-A1B-7B
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9 Molmo-A1B-7B
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6 Molmo-A1B-7B
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1 Molmo-A1B-7B
Video	Video-MME	33.7%	42.2%	52.1%	45.0 InternVL2-2B
Video	MLVU	40.6%	47.3%	55.2%	48.2 InternVL2-2B
Average	All shown	44.0%	51.0%	59.8%	—

SmoIVLM-256M also surpasses the 300× larger Idefics-80B per the paper key finding.

SmoIVLM-256M

44.0% avg

0.8GB RAM

SmoIVLM-500M

51.0% avg

1.2GB RAM

SmoIVLM-2.2B

59.8% avg

4.9GB RAM

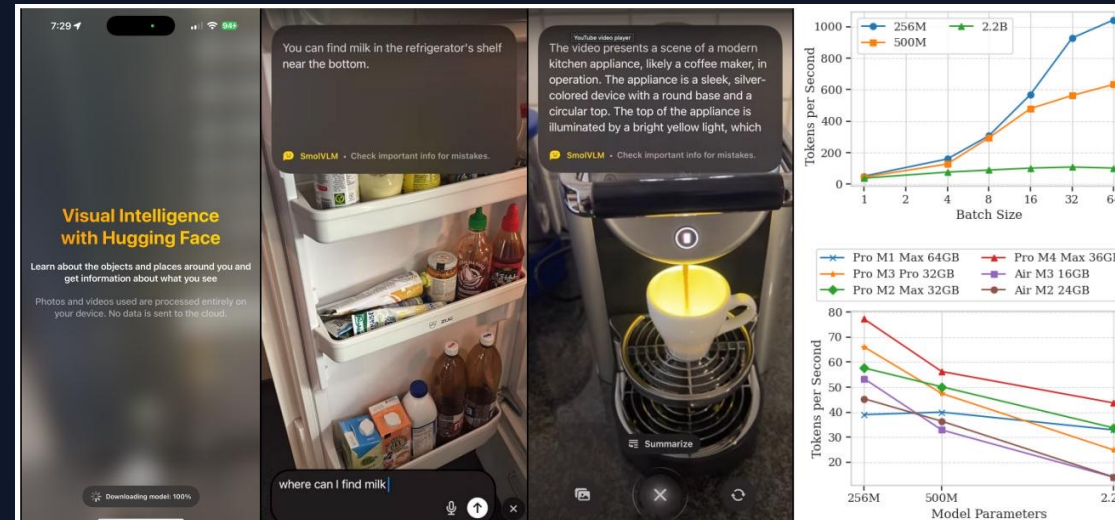
Molmo-A1B-7B

baseline RAM

27.7GB RAM

Source: SmoIVLM paper brief; benchmark rows shown are the rows provided in source_brief.md

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon



HuggingSnap app and throughput evidence from source figure

Local processing

photos + videos
no cloud dependency

~80 tok/s

Pro M1 Max 64GB
SmoIVLM-256M

~55 tok/s

MacBook Air M3 16GB
SmoIVLM-256M

~1000 tok/s

batch size 64
high-end devices

Throughput scales roughly linearly with batch size for the smallest model.

Key Takeaways: Design Principles for Efficient Multimodal Models

1

Balance compute

Match encoder size to LM capacity; smaller encoders can win for compact LMs.

2

Compress early

Use aggressive pixel shuffle for small models; $r=4$ reduces visual tokens $16\times$.

3

Scale context

Extend context with RoPE base tuning and mixed long/short fine-tuning.

4

Design for RAM

Sub-1GB inference is achievable: SmoVLM-256M uses 0.8GB at batch=1.

5

Open the stack

Release weights, datasets, code, and apps to accelerate reproducible edge AI.

Implication: efficient multimodal models are an architectural design problem, not only a scaling problem.

SmoVLM demonstrates competitive VLM capability under mobile and edge memory constraints.